

Json-extract — Q/A/S Split Architecture Analysis

Independent-output design review (questions.jsonl / answers.jsonl / solutions.jsonl)
Repository: ssachin998/Json-extract • Branch: arena/019fe0d7-json-extract • Reviewed:
2026-08-08

Status: ANALYSIS ONLY — no code modified

Verdict at a glance. Splitting Q / A / S at the *Gemini-call* level is **NOT SAFE**: the in-memory `chapter_records` state is load-bearing for 14+ invariants (run-18 hallucination guard, carry-forward expiry, section-boundary hard-reset, contamination guards, integrity sweep).

Splitting Q / A / S at the *write* level — after all in-pipeline reconciliations, writing three sidecar JSONL files keyed by a composite `q_id` with a `q_id_grade` — **IS SAFE** and is the recommended design. The existing `data/questions.jsonl` stays untouched, so the dashboard, the validator, and every existing consumer are unaffected.

A. Current Pipeline Trace

The pipeline today is a **shared-state, multi-pass, q_no-keyed merge engine**. The three Gemini passes (Q, A, S) do not run independently — they are co-ordinated loops over the same chapter in a single process, writing into one shared in-memory `chapter_records` dict keyed by `q_no`.

End-to-end flow (per chapter)

```
process_pdf() per chapter
■
■■■ Render chapter pages (pdftoppm, 150 DPI) → /tmp/{SUB}_ch{NNN}/page-NNN.jpg
■
■■■ For each window in section-aware or fixed 6-page windows:
■ ■
■ ■■■ Compute per-pass carry-forward (carry_by_pass["Q"|"S"])
■ ■■■ For (Q, A, S) in passes, IF active:
■ ■ ■
■ ■ ■■■ Build carry context from prev window (CONTEXT FROM PREVIOUS BATCH)
■ ■ ■■■ Call Gemini with prompt SCHEMA_PROMPT_{Q|A|S}
■ ■ ■■■ Extract _batch_meta and _figure_map control objects
■ ■ ■■■ (Q-only) run-18 GUARD: re-route discontinuous q_nos to orphans
■ ■ ■■■ (S-only) clip_pass_solutions() – strip redundant foreign tails
■ ■ ■■■ merge_question_records(chapter_records, items, stats) ← FILL-ONLY
■ ■ ■■■ Tag items with _prov (Q_PASS / A_PASS / S_PASS)
■ ■ ■■■ Update carry_by_pass via compute_carry() + enforce_carry_expiry()
■ ■ ■■■ (Q-only) detect_section_boundary() – sticky solutions-section toggle
■ ■ ■■■ Extract images; deterministic geometry (L1) → OCR geometry (L2)
■ ■ ■ → figure-map (L3) → full-page vision (L3b) → isolated-crop fallback
■ ■ ■■■ Update qn_source_pages[qn] (for later critique pass)
■ ■■■ (next window)
■
■■■ recover_orphans() – 5 rules to attach q_no-less fragments
■■■ Second / third / fourth image-claim passes
■■■ drain_failed_pages() – second-chance for recitation-blocked pages
■■■ chapter_integrity_sweep() – 5 deterministic proofs, quarantine suspect stems
■
■■■ targeted_retry() – up to 2 focused re-ask rounds, fill-only
■■■ rescue_incomplete_records() – page-focused last-ditch (max 12 calls)
■
■■■ drop_phantom_solution_only_records() – cross-chapter solution-spill proof
■■■ _export_gate_violations() – deterministic pre-export check
■■■ critique_and_repair_chapter() – run-19: model confirms/corrects flagged rows
■
■■■ build_final_question() – assemble final JSONL row,
■   qid = {SUB}-{chapter:03d}-{q_no:03d}
■■■ rewrite_questions_file() – atomic per-chapter write
■■■ write_chapter_file() – per-chapter JSONL
■■■ update state.json, chapters.json
```

On-disk artefacts

Path	Writer	Content
data/questions.jsonl	pipeline	Final merged dataset, one row/question
data/by_chapter/{chapter_id}.jsonl	pipeline	Same rows, per-chapter
data/chapters.json	pipeline	Chapter registry (incremental)
data/orphans.jsonl	pipeline	q_no-less fragments, with pdf_pages, pass, item
data/integrity_flags.jsonl	pipeline + sweep	Every deterministic sweep verdict
data/stem_conflicts.jsonl	merge engine	Both stem variants + verdict
data/still_incomplete_after_retry.jsonl	targeted retry	Post-retry gaps, with chapter_id
data/export_gate.jsonl	export gate	Gate violations
data/image_ownership.jsonl	image pass	Per-image ownership ledger
data/unmatched_images.jsonl	image pass	Figures with no claimant
data/dropped_*.jsonl	various	Drop ledger (phantom, anchorless, ...)

`qbank_validator.py` reads `data/questions.jsonl` and derives `q_no` from the `id` field via `q_no_of(row) = int(row["id"].rsplit("-", 1)[-1])`. It does not re-derive identity from anchors.

B. Current q_id Generation / Reconciliation Logic

B.1 Identity key

Identity = q_no (integer) within a chapter. It is the dict key of chapter_records — the only data structure that survives across batches, retries, and image-claim passes. The final id field is purely `f"{subject}-{chapter_no:03d}-{q_no:03d}"` (see `build_final_question`, line 3642).

q_no is **never recomputed from the source text** during normal operation. It is whatever integer Gemini returned in its JSON response.

B.2 The defenses that currently keep q_no honest

Defense	Where	What it does
_as_int_qno normalization	merge_question_records L3361 + process_pdf L5472	Forces int(64). Gemini sometimes returns "7" instead of 7
Per-field _prov tag	merge_question_records L3380	Every field remembers the pass that produced it; S/A can never populate a stem
Semantic stem guard	_stem_reject_reason L868	Rejects stems that open with explanation language or are contained in own solution
Stem-contamination discriminator	L882–L925	Two-pass check: opens-with-opener, token-share ≥ 0.8, reverse containment, le
Stem conflict resolver	merge_question_records L3430	Two stem variants for same q_no → coherence score picks the right one
Duplicate-stem sweep	chapter_integrity_sweep L1100	Identical stems on two records → coherence loser stripped
Contamination quarantine	L1190	Suspect stem kept (not deleted) but flagged _stem_suspect_reason
Stale-carry guard #1	enforce_carry_expiry L2941	Carry dies after 3 batches unresolved; q_no banned for the rest of the chapter
Stale-carry guard #2	detect_section_boundary L2905	First "Solution to Question N:" hard-resets all carry
Q-pass GUARD (run-18)	process_pdf L5495	Q-pass with discontinuous q_nos not extending chapter max → orphans
Section planner	build_section_windows L1807	Questions vs solutions windows by text-layer probe
_OCR_CONTENT_OWNER	L717	For OCR fallback only: matches option tokens to existing record
Wrong-owner guards	_foreign_option_line, _solution_fragment_line L5021	Reject solutions beginning with "Option X:" of an option the record doesn't own
Cross-chapter phantom drop	drop_phantom_solution_only_records L5022	Chapter solution-only record duplicating an earlier chapter's q_no → drop
Anchorless drop	L5990	Rows with no stem + no options + no solution → dropped
Q-coverage safety net	L5298	Page Q-pass never ran on → OCR check, run Q anyway
OCR-anchored figure geometry	claim_block_images_ocr L4170	Question-side figures claimed by OCR of rendered page
figure_map	process_pdf L5597	Gemini's own _figure_map attaches figures to q_nos (used only after determinis

B.3 What "trusting q_no" really means today

- The pipeline trusts that the Gemini Q-pass response carries the printed q_no as an integer.
- The Gemini A-pass response carries q_no as an integer (from the printed key table).
- The Gemini S-pass response carries q_no as an integer (from "Solution to Question N:" header).
- Each pass, on the same set of pages, returns roughly the same set of q_nos.
- q_no integer identity survives normalization (`int(...)`).
- Hallucinations are caught by the run-18 GUARD: a Q-pass item with a q_no not yet known to the chapter, not part of a large connected run, and not visible via OCR on the page, is sent to orphans.

B.4 Where q_no comes from in the source

Source	Helper	Mechanism
Question-stem heading	qns_printed_on_page, question_heading	Regex on page / pypdf text visitor
Answer-key table row	KEY_TABLE_PROBE_RE + ANSWER_KEY_ROW_RE	Regex in markdown pipe table
"Solution to Question N:" heading	SOLUTION_PROBE_RE / SOLUTION_MARKER_RE / SOLUTION_DUMP_HDR_RE	Regex on page / Solution New
Carry-forward	compute_carry	Uses LAST q_no from previous batch's items + _batch_meta
Orphan recovery rule 1	recover_orphans	Self-label "Solution to Question N:" inside fragment
Orphan recovery rule 2	recover_orphans	carry_q_no from the carry-forward state
Orphan recovery rule 3	recover_orphans	last_qn_in_batch (highest q_no in same window) with "missing that field" check
Orphan recovery rule 4	recover_orphans	Sole stemless record (Gap-1) — only Q-pass fragments
Orphan recovery rule 5	recover_orphans	Verified duplicate (consume, no ownership guess)

C. Exact Risks of Making Q / A / S Truly Independent

The proposal is to write each pass's output to its own JSONL file (`questions.jsonl`, `answers.jsonl`, `solutions.jsonl`) and merge later using a `q_id`. Below are the concrete risks, each tied to a code reference.

C.1 Loss of co-occurrence-based defenses

`recover_orphans` (L2993) today merges Q-pass and S-pass orphans into the SAME `chapter_records`. Rules 1, 3, 4 depend on observing the three streams simultaneously. After split, an orphan in `questions.jsonl` has no knowledge of what `solutions.jsonl` produced. Rule 3 and Rule 4 cannot work.

C.2 Loss of contamination guard

`_stem_reject_reason` (L868) and `merge_question_records` (L3411) drop a would-be stem by comparing it against the receiving record's `solution_text`. After split, the Q-pass file has no `rec` (solution text) to compare against — a contaminated stem cannot be flagged.

C.3 Hallucination guard (run-18) breaks

`process_pdf` L5495 relies on `chapter_records` (in-progress set), `known_max`, `trusted_qnos` from connected runs, and OCR of the rendered page. After split, the Q-pass file is written with no `chapter_records`. The OCR-verification step is per-window, but the "near `known_max`" branch requires `known_max` from a chapter already partially extracted.

C.4 Carry-forward is lost

`compute_carry` (L2555) + `build_carry_context` (L2630) inject partial-stem context into the SAME next Gemini call. After split, the Q-pass call for page 11 has no way to know what Q23's partial stem was, because the carry is in-memory, not persisted.

C.5 Stale carry, banned `q_nos`, and section boundary all break

`enforce_carry_expiry` (L2941), `detect_section_boundary` (L2905), `carry_by_pass` are all per-pass, per-chapter, in-memory. The cross-section fix (audit-cited) is this shared state. After split, the boundary hard-reset that prevents stale question-side carry from meeting its number again in the Solutions section is gone.

C.6 `q_no` type confusion returns

`merge_question_records` L3361 + `process_pdf` L5472 normalize at the merge boundary today. After split, if Q-pass writes "23" (string) and S-pass writes 23 (int), the merge has to align types. Case 4 from the spec becomes a real bug instead of a one-line `int()` cast.

C.7 'Q-pass misses Q23 but S-pass finds it' is no longer recoverable

Today, `recover_orphans` Rule 1 (self-label) and Rule 3 (carry continuation) attach the S-pass fragment to `chapter_records`. After split, the Q-pass file has no Q23 at all. The merge sees 'Q23 absent from questions, present in solutions' and has no signal to disambiguate missed vs hallucinated vs foreign without re-reading the source PDF.

C.8 'A-pass hallucinates Q23' cannot be filtered

The Q-pass hallucination GUARD only fires on the Q-pass file. The A-pass and S-pass have no equivalent guard. After split, the A-pass file and S-pass file don't know the chapter's 'known' `q_no` set.

C.9 'Printed question number is not detected' loses the only fallback

Today, when a question's printed number is not detected, the Q-pass item comes back with `q_no = null`, goes to orphans, and `recover_orphans` Rule 4 ('sole stemless record') reassigns it. The S/A passes still write the `q_no` they see, anchoring the recovery. After split, a Q-pass item with `q_no = null` has no way to know the S/A pass's `q_no` set.

C.10 Section boundary + carry hard-reset = lost

`process_pdf` L5418 hard-resets all carry context when the first S section is detected — the single most important defense against cross-section contamination. If the Q-pass file is written before the S section is even seen, this defense is no longer possible.

C.11 Image ownership gets harder

`claim_figure_map_images` (L4582), `full_page_vision_ownership` (L4430), `_record_image_ownership` (L4332) attribute images by `q_no + slot`, claimed by the chapter records' `q_no` set. After split, image ownership is decoupled from question identity — a figure attached to Q23 in the S-pass's `_figure_map` cannot be matched to Q23 in the Q-pass file unless `q_no` keys agree on type and value.

C.12 Cross-chapter phantom solution drop requires chapter-level view

`drop_phantom_solution_only_records` (L5022) reads `data/questions.jsonl` for prior chapters. After split, it must read three files per chapter instead of one. It still works, but the check is now on three files.

C.13 Chapter-end second-pass image claiming breaks

`process_pdf` L6000 ('SECOND PASS image claiming') re-claims unmatched images after all batches because chapter records are then complete. After split, the Q-pass file is written before S-pass; the S-pass has no chance to re-claim images whose owner was a question extracted after the image's page.

C.14 Quota accounting and resume semantics

`MAX_CALLS_PER_DAY = 480 + state['calls_today']`. Today, a quota exit mid-chapter throws `sys.exit(0)` and resume picks up via `state + the page ledger`. After split, the Q-pass file may be complete but `questions.jsonl` empty until S-pass also runs. Resume semantics are harder — the master is no longer one file that can be atomically rewritten per-chapter.

C.15 Audit trails diverge

`data/integrity_flags.jsonl` is one shared ledger today (foreign fragments, stem rejections, contamination). After split, each pass can only log its own contamination. The merge step needs to re-aggregate them, and chapter-level flags (suspect stem, foreign solution dump) cannot be detected until the merge step.

C.16 Validator depends on `id = {SUB}-{NNN}-{QQQ}`

`qbank_validator.py` L187 derives `q_no` from `row['id'].rsplit('-', 1)[-1]`. After split, there is no single `id` until merge. The independence is illusory for the validator — the merge step IS effectively building the current `questions.jsonl` from three files.

C.17 Carry rescue / record rescue for images is impossible after split

`claim_block_images_ocr` (L2) and `full_page_vision_ownership` (L3) need `chapter_records` to know which `q_nos` exist. After split, the L2/L3 claimers can't run for S-pass-side figures whose owner was a Q-pass-only item not yet known.

D. Proposed q_id Reconciliation Architecture

The proposed architecture **preserves the shared-state design during the chapter and splits only at the write step**. The Q/A/S calls still share chapter_records (so all the guards keep working), but at build_final_question time the chapter is also serialized to three sidecar JSONL files. Crucially, every row is tagged with deterministic anchors so the merge step (or a downstream consumer) can re-verify identity without trusting the model's q_no.

D.1 Core principle: identity is a tuple, not an integer

```
q_id = {
  chapter_id:  "PSY-001",          # from TOC, immutable
  subject:     "PSY",
  chapter_no:  1,                  # from TOC
  q_no:        23,                 # model's integer, PROVISIONAL
  q_no_anchors: [...],             # see D.4
  q_id_grade:  "RESOLVED" | "RESOLVED_ANCHORED" |
               "PROVISIONAL" | "UNRESOLVED"
}

# The q_id_grade is the only thing a consumer should trust
# when joining. The integer q_no is a HINT, not the identity.
```

D.2 The four confidence grades

Grade	Meaning	Allowed actions
RESOLVED_ANCHORED	At least 2 independent anchors agree on the q_no AND/OR text finalizer	Auto merge, question integrity header
RESOLVED	2 anchors agree (one is a "Solution to Question N" header, the other is a "Question N" header), OR a single anchor + a 5-item confidence	Auto merge, question integrity header
PROVISIONAL	Only the model's q_no, no second anchor, no confidence	Auto merge with a low_confidence=true flag; consumer must verify or c
UNRESOLVED	No anchor at all, or two anchors disagree, or q_no not in TOC	Do NOT write to unresolved.jsonl with full provenance. Hallu

D.3 What changes in the pipeline (the minimum that works)

Recommendation: DO NOT split the three passes into independent processes. Instead, split only the **WRITES** at the end of the chapter. The Q/A/S calls still share chapter_records (so all the guards keep working), but at build_final_question time the chapter is also serialized to three sidecar JSONL files in addition to questions.jsonl.

Concretely, at the end of process_pdf (after build_final_question and the final rewrite_questions_file), add:

```
write_three_split_files(chapter_id, chapter_records,
                        image_files_by_q, qn_source_pages)
```

This writes three files:

- **data/split/{chapter_id}.questions.jsonl** — one row per resolved question, fields: q_id, chapter_id, subject, chapter_no, q_no, q_no_anchors, q_id_grade, question_text, options, tables, question_images, source_pages
- **data/split/{chapter_id}.answers.jsonl** — one row per question WITH a correct_option, fields: q_id, q_no, correct_option, correct_option_prov (which pass filled it), confidence
- **data/split/{chapter_id}.solutions.jsonl** — one row per question WITH a solution, fields: q_id, q_no, solution_text, tables, solution_images, solution_prov, source_pages

Crucially:

- Every row carries `q_no_anchors` (see D.4).
- A Q23 that exists in `solutions.jsonl` but NOT in `questions.jsonl` (Case 2) is logged to `unresolved_qids.jsonl` with reason 'missing_question_for_solution'. The user-facing consumer decides whether to synthesize a placeholder question or show 'Solution found, question missing — review'.
- A Q23 that exists in `questions.jsonl` but NOT in `solutions.jsonl` is just a missing solution — already handled by the export gate.
- A Q23 that exists in all three with different grades is the normal case.

D.4 The `q_no_anchors` field — the deterministic reconciliation evidence

For every merged record, compute and persist ALL of the following that are true:

```
q_no_anchors = {
  "model_q_no": 23, # from any pass that set it
  "model_q_no_provs": ["Q_PASS", "S_PASS"], # which passes produced a q_no
  "model_q_no_disagree": false, # any disagreement?
  "printed_stem_match": { # from question_headers_on_page
    "page": 12, "y_baseline": 423.5, "header_text": "23."
  } or null,
  "printed_solution_header_match": { # from solution_headers_on_page
    "page": 47, "y_baseline": 200.1, "header_text": "Solution to Question 23:"
  } or null,
  "answer_key_row_match": { # from ANSWER_KEY_ROW_RE
    "page": 22, "row": "| 23 | B |"
  } or null,
  "ocr_stem_match": { # from ocr_page_anchors
    "page": 12, "q_no": 23
  } or null,
  "ocr_solution_header_match": { # from ocr_page_anchors
    "page": 47, "q_no": 23
  } or null,
  "neighbor_run": { # connected run from run-18 GUARD
    "size": 12, "first": 18, "last": 29, "near_chapter_max": true
  } or null,
  "carry_forward_origin": { # from compute_carry
    "from_window": 3, "cut_part": "options", "resolved_in_window": 4
  } or null,
  "section_position": { # from block_headers_on_page
    "kind": "question", "y_baseline": 423.5, "page": 12
  } or null,
}
```

This is a **provenance vector**, not a single key. The reconciliation grade is computed from it:

```
def grade_from_anchors(anchors):
  matches = sum([
    bool(anchors['printed_stem_match']),
    bool(anchors['printed_solution_header_match']),
    bool(anchors['answer_key_row_match']),
    bool(anchors['ocr_stem_match']),
    bool(anchors['ocr_solution_header_match']),
  ])
  if anchors['model_q_no_disagree']: return 'UNRESOLVED'
  if matches == 0: return 'UNRESOLVED'
  if matches >= 2 and (anchors['printed_stem_match']
    or anchors['printed_solution_header_match']):
    return 'RESOLVED_ANCHORED'
  if matches >= 2 or (matches == 1 and (anchors['neighbor_run']
```

```

    return 'RESOLVED'
    return 'PROVISIONAL'

    or anchors['carry_forward_origin']))):
```

D.5 The reconciliation gate (new, in the write split step)

After all merges, retries, rescues, sweeps, and critiques, before writing the three files:

```

reconcile_qids(chapter_id, chapter_records, qn_source_pages,
               pdf_path, page_files)
-> writes unresolved_qids.jsonl, updates chapter_records with _q_id_grade
-> returns (resolved_count, provisional_count, unresolved_count)
```

Records that come out UNRESOLVED:

- Get their q_no set to null in the merged record
- The question_text (if present) is preserved
- The record is written to unresolved_qids.jsonl with the full q_no_anchors provenance
- It is NOT written to split/{chapter_id}.questions.jsonl (so the split file is empty for that q_no)
- A solutions.jsonl row for the same q_no (if it had a solution) is also dropped to unresolved_qids.jsonl to prevent a 'ghost answer' from shipping
- The original questions.jsonl row's q_no becomes a 'ghost' with q_id_grade = 'UNRESOLVED' and q_id_anchors populated — the consumer sees it explicitly

D.6 Case-by-case handling

Case	Current behavior	After split-with-anchors	Outcome
1. Q=23, A=23, S=23 merged into flags		All anchors agree → RESOLVED_ANCHOR	Three files get a row, grades agree
2. Q-pass misses Q23, S-pass prints Solution to Q23, A-pass prints Q23 solution		Q-pass for Q23 and Q23 solution resolved, S-pass for Q23 and Q23 solution resolved	Right here, neither has solution, question for flag
3. Q-pass says Q23, S-pass says Q24, A-pass says Q23		Q-pass for Q23 and Q23 solution resolved, S-pass for Q23 and Q23 solution resolved, A-pass for Q23 and Q23 solution resolved	Right here, neither has solution, question for flag
4. Q-pass "23" (string) as _pass_q23 (int) normalizes		Anchor grading normalizes both via int()	Right here, neither has solution, question for flag
5. A-pass hallucinates Q23 (w/printed chapter)		A-pass for Q23 and Q23 solution resolved, S-pass for Q23 and Q23 solution resolved, A-pass for Q23 and Q23 solution resolved	Right here, neither has solution, question for flag
6. Two possible question for resolution foreign		Same as the first file as the fragment is, in the second file, the fragment is, in the third file, the fragment is	Right here, neither has solution, question for flag
7. Question continues across two pages		The two halves of Q23 get different _prov	Right here, neither has solution, question for flag
8. No printed q_no detected, but pass Rule 4 (split not unless second)		Added a new row, merge uses the carry	Right here, neither has solution, question for flag

D.7 What does NOT change

- The Q/A/S Gemini calls remain in the SAME process_pdf loop, with the SAME chapter_records shared state.
- The carry, the section boundary, the run-18 GUARD, the integrity sweep, the targeted retry, the rescue, the critique pass — all keep their current implementations.
- qbank_validator.py does not change — it still reads data/questions.jsonl and derives q_no from id.
- The atomic rewrite of questions.jsonl per chapter stays the source of truth for the existing dashboard/zip.
- The three split files are ADDITIONAL outputs, not replacements.

E. Required Changes

E.1 chapter_records schema additions

Add a record-level field (backward-compatible with today's per-field `_prov`):

```
chapter_records[qn]["_q_id_anchors"] = {
    "model_q_no": 23,
    "model_q_no_provs": [],
    "model_q_no_disagree": False,
    "printed_stem_match": ...,
    "printed_solution_header_match": ...,
    "answer_key_row_match": ...,
    "ocr_stem_match": ...,
    "ocr_solution_header_match": ...,
    "neighbor_run": ...,
    "carry_forward_origin": ...,
    "section_position": ...,
    "provenance_notes": [], # list of short strings
}
chapter_records[qn]["_q_id_grade"] = "PROVISIONAL" # set by reconcile_qids

# The _prov dict is already there for per-field provenance.
# _q_id_anchors is a record-level concept.
```

E.2 Hallucination guard (run-18 GUARD) — what must change

The current guard at process_pdf L5495 is already what we need at the Q-pass stage. What changes:

- After merge, every Q-pass-sourced record that passed the GUARD still gets `q_no_anchors.model_q_no_provs` += ['Q_PASS'] even if the `q_no` wasn't verified by OCR. The GUARD's 'trusted' set becomes a `neighbor_run` anchor.
- The GUARD's 'unconfirmed → orphan' path is preserved. Orphans that go to `orphans.jsonl` already carry pass and reason. We add `q_no_anchors` to each orphan.

For the A-pass and S-pass, an equivalent guard is needed:

- A-pass: every row's `q_no` must be in the chapter's 'known `q_nos`' (after Q-pass has run for the same pages) OR be present in the printed answer-key table on those pages. Otherwise → UNRESOLVED with reason 'answer_q_no_not_in_printed_key'.
- S-pass: every row's `q_no` must be in the chapter's 'known `q_nos`' OR be a printed 'Solution to Question N:' header. Otherwise → UNRESOLVED with reason 'solution_q_no_not_in_printed_header'.

This guard runs DURING the merge, not after — using `qn_source_pages[qn]` (already populated) and the result of a new `qns_printed_on_page_v2` (which accepts an arbitrary `q_no` set).

E.3 Carry-forward logic

- The carry state stays in-memory, per-pass, per-chapter. It does not get serialized. The split files only reflect what the chapter-final `chapter_records` contains. The carry is a runtime convenience for the SAME Gemini call; the merge step does not re-derive it.
- The split file's `q_no_anchors.carry_forward_origin` is populated from `compute_carry`'s output, which the current code already tracks. No new code; just emit the existing carry state as a JSON-friendly field.

E.4 Retry logic — what must change

- `targeted_retry` and `rescue_incomplete_records` continue to call Gemini with their existing prompts. No change to call sites.
- After a retry round, the merge re-applies `_stem_reject_reason` (L3411) and `_solution_fragment_foreign` (L3446) — unchanged.
- NEW: after every retry round, the `q_no_anchors` are updated. The model's `q_no` from the retry response is added to `model_q_no_provs` (e.g. ['Q_PASS', 'Q_RETRY']). If the retry's `q_no` differs from the original, `model_q_no_disagree` = True and the grade drops to UNRESOLVED until the next pass confirms.

E.5 Integrity sweep changes

In `chapter_integrity_sweep` (L1075):

- The `duplicate-stem strip` (step 1) and `contamination quarantine` (step 5) keep their current behavior.
- NEW: at the end of the sweep, `reconcile_qids(chapter_records, ...)` is called. This populates `_q_id_anchors` and `_q_id_grade` for every record. The grade becomes a sweep output, and `_q_id_grade == 'UNRESOLVED'` records are added to the export gate as `unresolved_qid` violations.

E.6 Drop / phantom check

- `drop_phantom_solution_only_records` (L5022) becomes a `q_id_grade` check: a record whose only content is a solution AND whose `q_id_grade` is UNRESOLVED AND whose solution duplicates a prior chapter's `q_no` is dropped. Records that are RESOLVED keep their solution.
- `drop_anchorless_records` (L5980) becomes a stricter check: drop only when `_q_id_grade == 'UNRESOLVED'` AND no anchor is present. A PROVISIONAL record with one weak anchor is kept and flagged.

E.7 Export gate additions

`_export_gate_violations` (L4945) gains a new violation kind:

```
("unresolved_qid", qn,
 f"q{qn} could not be confirmed against the source PDF — "
 f"{anchors['provenance_notes']}")
```

The export gate's 'CLEAN' verdict now requires `unresolved_qid == 0`.

F. Example of the Resulting Data Flow for Q23

Suppose the source chapter has Q23 on PDF page 12, with options A-D on page 12, and a "Solution to Question 23:" header on PDF page 47.

F.1 Day-to-day case (Case 1, all three pass)

```
[Q-pass] Gemini returns: {q_no: 23, question_text: "...",
                        options: {A:..., B:..., C:..., D:...}}
[S-pass] Gemini returns: {q_no: 23, solution_text: "...", tables: [...]}
[A-pass] Gemini returns: {q_no: 23, correct_option: "B"} # from key on page 22
```

After merge + integrity sweep, the record's `_q_id_anchors` look like:

```
chapter_records[23] = {
  "q_no": 23,
  "question_text": "...",
  "options": {"A": ..., "B": ..., "C": ..., "D": ...},
  "correct_option": "B",
  "solution_text": "...",
  "tables": [...],
  "_prov": {"question_text": "Q_PASS", "options": "Q_PASS",
            "correct_option": "A_PASS", "solution_text": "S_PASS"},
  "_q_id_anchors": {
    "model_q_no": 23,
    "model_q_no_provs": ["Q_PASS", "A_PASS", "S_PASS"],
    "model_q_no_disagree": False,
    "printed_stem_match": {"page": 12, "y_baseline": 423.5, "header_text": "23."},
    "printed_solution_header_match": {"page": 47, "y_baseline": 200.1,
                                       "header_text": "Solution to Question 23:"},
    "answer_key_row_match": {"page": 22, "row": "| 23 | B |"},
    "neighbor_run": {"size": 26, "first": 1, "last": 26,
                     "near_chapter_max": True},
    "section_position": {"kind": "question", "page": 12, "y_baseline": 423.5},
    "provenance_notes": ["Q_RETRY unchanged", "stem coherence 0.78"],
  },
  "_q_id_grade": "RESOLVED_ANCHORED",
}
```

Three files are written:

```
# data/split/PSY-001.questions.jsonl (one line):
{
  "q_id": "PSY-001-023",
  "chapter_id": "PSY-001", "subject": "PSY", "chapter_no": 1, "q_no": 23,
  "q_id_grade": "RESOLVED_ANCHORED",
  "q_no_anchors": { "...": "..."},
  "question_text": "...",
  "options": {"A": "...", "B": "...", "C": "...", "D": "..."},
  "tables": [], "question_images": [],
  "source_pages": [12]
}

# data/split/PSY-001.answers.jsonl (one line):
{
  "q_id": "PSY-001-023", "chapter_id": "PSY-001", "q_no": 23,
  "correct_option": "B", "correct_option_prov": "A_PASS",
  "q_id_grade": "RESOLVED_ANCHORED",
  "source_pages": [22]
}

# data/split/PSY-001.solutions.jsonl (one line):
```

```
{
  "q_id": "PSY-001-023", "chapter_id": "PSY-001", "q_no": 23,
  "solution_text": "...", "tables": [...], "solution_images": [],
  "solution_prov": "S_PASS",
  "q_id_grade": "RESOLVED_ANCHORED",
  "source_pages": [47]
}
```

A consumer joins on `q_id` and ships the final row. The existing `data/questions.jsonl` row is also still written and is the source of truth for the existing dashboard.

F.2 Case 2 (Q-pass misses, S-pass finds)

```
[Q-pass] returns nothing for q_no=23
[S-pass] returns {q_no: 23, solution_text: "...", tables: [...]}
[A-pass] returns {q_no: 23, correct_option: "B"} # printed key table row
```

After merge + recovery:

- `chapter_records[23]` does NOT exist (Q-pass missed it).
- S-pass's fragment is filtered into orphans (`q_no` not in `chapter_records`).
- A-pass's row is also filtered into orphans.
- `recover_orphans` Rule 1 (`SOLUTION_TO_Q_RE`) finds 'Solution to Question 23:' inside the S-pass fragment → creates `chapter_records[23]` with `solution_text` filled.
- A-pass's `correct_option` is merged fill-only into the new record.

The record's `q_no_anchors`:

- `model_q_no`: 23
- `model_q_no_provs`: ['S_PASS', 'A_PASS'] (Q never produced a `q_no`)
- `printed_solution_header_match`: ✓ (from S-pass self-label + `solution_headers_on_page` re-scan)
- `answer_key_row_match`: ✓
- `printed_stem_match`: null ← THIS is the missing anchor
- `provenance_notes`: ['Q-pass produced no item; created from S-pass self-label + A-pass key row']
- `q_id_grade` = RESOLVED (two anchors, no stem match)

Split files written:

- `questions.jsonl`: NO row for `q_no=23` (the question stem is genuinely missing)
- `answers.jsonl`: row for `q_no=23` with `correct_option`: 'B', `q_id_grade`: 'RESOLVED', `provenance_notes`: ['no_question_text']
- `solutions.jsonl`: row for `q_no=23` with the full solution, `q_id_grade`: 'RESOLVED', `provenance_notes`: ['no_question_text']
- `unresolved_qids.jsonl`: one entry with `kind`: 'missing_question_for_solution'

Critical guarantee: no ghost question is shipped. The current pipeline would have shipped Q23 with an empty `question_text` and the export gate would have flagged `missing_stem` — which is the SAME end state, but the new design makes it visible in the split files and the unresolved ledger, so a downstream tool can act on it without re-running the chapter.

G. Edge Cases and Failure Handling

G.1 The 8 cases from your spec

Case	Behavior in proposed design
1. Q/A/S all find Q23 consistently	RESOLVED_ANCHORED; all three files get a row; consumer joins cleanly
2. Q misses, A/S find Q23	RESOLVED; question file empty for Q23; unresolved_qids.jsonl entry 'missing_question_for_solution'; downstream
3. Q says Q23, S says Q24, solution matches Q23	model_q_no_disagree_Q23 = True; q_id_grade = UNRESOLVED; both q23 and q24 marked suspect; targeted re-ask
4. Q "23", S 23	Normalized via int(); model_q_no_provs records both; no type mismatch at join
5. A hallucinates Q23	A-pass's item has model_q_no=23 but no answer_key_row_match → UNRESOLVED; row goes to unresolved_q
6. Two possible questions for one solution	solution_fragment_foreign detects; model_q_no_disagree triggers; both candidates UNRESOLVED; targeted re
7. Question continues across two pages	Carry origin preserved; q_no_anchors.carry_forward_origin populated; q_id_grade = RESOLVED
8. No printed q_no, stem identified	printed_stem_match: null, but neighbor_run and sole_stemless_record provenance populated. RESOLVED if exa

G.2 Additional edge cases

Q-pass item with q_no: null, OCR finds it

Q-pass GUARD routes to orphans. recover_orphans Rule 4 or Rule 1 gives it an owner. After merge, the q_no is set. The q_id_anchors reflect 'OCR found q_no on page N at position (x,y)'. Grade = RESOLVED.

S/A find Q23, but Q-pass never sees Q23 because pages 11-12 are mislabeled as Solutions by the section planner

S/A files both have Q23. The reconciliation step queries the Q-coverage safety net (window_has_question_content). If page 12 has a question-stem heading (OCR confirms), Q23 is forced into questions file with q_id_grade = RESOLVED and provenance_notes: 'synthesized_from_s_and_a_passes_q_coverage_safety_net'. Otherwise, UNRESOLVED.

Cross-chapter spill (Run-14 phantom class)

The solution-only record has q_id_grade = UNRESOLVED AND solution similarity to a prior chapter's q_no > 50% → drop_phantom_solution_only_records drops it. The prior chapter's split files are not touched.

Recitation-blocked solution page

S-pass skipped; OCR recovers via _recover_ocr_solution_headers. Recovered solution carries provenance_notes: 'ocr_recovery_no_visual'. q_id_grade = RESOLVED because the 'Solution to Question 23:' header is in the OCR text. Grade drops to PROVISIONAL if OCR confidence is low. Consumer decides to skip or display.

Hallucinated S-pass with no printed header

printed_solution_header_match: null, ocr_solution_header_match: null, no model_q_no_provs from Q/A, and no neighbor_run or carry_forward_origin → q_id_grade = UNRESOLVED. Record is written to unresolved_qids.jsonl only. No silent ghost.

Recitation block in Q-pass for q23

S/A may still have solution/answer. Q-pass page in state['failed_pages'] with pass: 'Q'. After drain, if it still fails, the record lands as chapter_records[23] = { solution_text: ..., correct_option: 'B', _q_id_grade: 'RESOLVED' (or PROVISIONAL) with provenance_notes: 'Q-pass never produced; drained S+A only' }. The questions file is empty for q23; unresolved_qids.jsonl has the entry.

Targeted retry produces a contradicting q_no

After merge, model_q_no_disagree = True and q_id_grade = UNRESOLVED. The record is not written to any split file until the next chapter-end pass re-asks.

Validator and questions.jsonl are the source of truth

The new split files are downstream. A user's existing dashboard reads data/questions.jsonl. The new files are for the new 'merging' step or a future consumer. qbank_validator.py is unchanged.

Crash mid-write of split files

A partial split file is a known-bad artifact. Use the same per-chapter atomic-rename pattern as rewrite_questions_file (L6407). A consumer that sees an incomplete split file knows the chapter is unfinished.

Schema change to chapter_records

The _q_id_anchors field is added at the end. Backward-compatible: old fields untouched, new field is optional. Recovery mode (fill_only=True) preserves it.

Quota brake mid-chapter

Today's design exits via sys.exit(0) after saving state. The split files are NOT written yet (they're written at chapter end, after all retries). The next run picks up via chapters_done and redoes the chapter from scratch. No regression — this is identical to today's behavior.

Image ownership across split files

Images are NOT in the split files. They stay in data/by_chapter/{chapter_id}.jsonl and assets/questions/... A future image-aware consumer joins by q_id on the chapter's image_files_by_q (already a chapter-level dict). The split files just expose the q_id→qn mapping for join keys.

G.3 The most subtle edge case: out-of-order completion

Because the Q/A/S passes run inside one process_pdf loop, they always complete for a chapter in order. But if you ever parallelize the three passes across machines, the A-pass and S-pass files may exist before the Q-pass file does. The reconciliation must be tolerant of this:

- The unresolved_qids.jsonl must support incremental writes (one entry per q_id as soon as a single pass records it, before all three agree).
- A consumer that reads split/PSY-001.questions.jsonl before split/PSY-001.answers.jsonl exists should treat absent q_nos as PROVISIONAL (with a 'waiting for other passes' note), not as UNRESOLVED. This requires a manifest.json per chapter listing which files have been written.

The simpler design is to **never parallelize** the three passes; the split is only at the write step. This is what the proposal recommends.

H. Recommended Implementation Plan

The plan is **strictly additive** — every step is reversible. No step removes today's behavior. The order is chosen so each step is testable on the trial book before the next begins.

Phase 0 — Investigation / scoping (no code change)

- Confirm the spec. I have presented Sections A–G. Do not move to Phase 1 until you confirm:
 - The split is at the WRITE step (not at the Gemini call step).
 - The split files are ADDITIVE to today's data/questions.jsonl (which remains the source of truth).
 - The new unresolved_qids.jsonl is acceptable as a sidecar.
 - The grade taxonomy (RESOLVED_ANCHORED / RESOLVED / PROVISIONAL / UNRESOLVED) is the contract.
- Define the schema for the three split files as a JSON Schema or pydantic model, so the consumer side is unambiguous. Final review before any code.

Phase 1 — Observation only (no behavior change)

- Add q_no_anchors collection as a passive observer. In process_pdf, after every merge_question_records call, populate chapter_records[qn]['_q_id_anchors'] from the in-memory state (model_q_no_provs, printed_stem_match from question_headers_on_page, etc.) WITHOUT changing any merge logic or guards. Write a debug JSONL data/observed_qid_anchors.jsonl per chapter. Confirm on the trial book that:
 - The anchors are populated correctly.
 - The grades (RESOLVED_ANCHORED etc.) match what a human would assign.
 - The observer never alters a record.
- Compare to today's data. Diff the observed anchors against today's behavior on a known chapter. Any case where the new design would have flagged a record as UNRESOLVED while today shipped it cleanly is reviewed manually. A test set is built.

Phase 2 — Reconciliation gate

- Implement reconcile_qids(chapter_records, ...) in qbank_pipeline.py. It is called at the end of process_pdf, AFTER all merges, retries, rescues, sweep, critique, but BEFORE build_final_question. It populates _q_id_anchors and _q_id_grade for every record. Records that are UNRESOLVED are flagged in the export gate (new kind unresolved_qid).
- Run on the trial book. Verify that:
 - The number of UNRESOLVED records is small and matches the cases today that are already exported with a gate flag.
 - The number of PROVISIONAL records is non-zero but tractable.
 - The number of RESOLVED_ANCHORED records is the majority.
- Tune the grading thresholds. If too many records are PROVISIONAL (because the OCR fallback fires often), add a second OCR pass or accept PROVISIONAL as the default and let the consumer filter.

Phase 3 — Write split files

- Implement `write_three_split_files(chapter_id, chapter_records, image_files_by_q, qn_source_pages)` in `qbank_pipeline.py`. Called at the end of `process_pdf`, AFTER the `questions.jsonl` rewrite. Writes the three files atomically (per-chapter temp file + rename).
- Implement `write_unresolved_qids(chapter_id, chapter_records)`. Called by `reconcile_qids` BEFORE the split writes. Writes only records with `q_id_grade == 'UNRESOLVED'`.
- Implement the run-18 GUARD for A-pass and S-pass. For A-pass, every row's `q_no` must be in the chapter's 'known `q_nos`' or be in the printed key table on the page. For S-pass, every row's `q_no` must be in the chapter's 'known `q_nos`' or be a printed 'Solution to Question N:' header. This runs during the merge, using the same `qns_printed_on_page`-style helpers but extended to accept an arbitrary `q_no` set.
- Re-run on the trial book. Verify the three split files have the right shape. Verify `unresolved_qids.jsonl` is small and explained.

Phase 4 — Consumer side

- Add a small `qid_join.py` tool (or extend the existing `qbank_validator.py` with an opt-in flag) that:
 - Reads the three split files for a chapter
 - Joins on `q_id`
 - Emits a merged JSONL (e.g. `data/joined/PSY-001.merged.jsonl`) with one row per question
 - Honors `q_id_grade`: UNRESOLVED rows are emitted with `q_id_grade` field but the rest of the row may be partial
 - Surfaces `unresolved_qids.jsonl` as a separate report
- Document the join in `PIPELINE_WORKFLOW.md` (or a new doc) — what the consumer can rely on, what it cannot.

Phase 5 — Optional dashboard surface

- If the user wants to see `q_id_grade` and `unresolved_qids.jsonl` on the dashboard, add a small route to `app.py` that lists unresolved `q_ids` per chapter.
- Do not change the existing `/run`, `/validate`, `/fix`, `/recover` routes. The split files are read-only outputs.

Phase 6 — Rollout gate

- Run on the full trial book. Verify:
 - `data/questions.jsonl` is byte-identical (or near-identical) to today.
 - `data/split/*.jsonl` files exist and are well-formed.
 - `data/unresolved_qids.jsonl` is small.
 - No new export gate violations.
 - Validator output is unchanged.
- Run on a SECOND book (e.g. a different subject PDF). Confirm the same logic holds. If UNRESOLVED count is high on a different layout, tune the anchors.

Phase 7 — Switch the consumer (LAST step, gated on the above)

- If and only if the user explicitly decides to consume the three split files instead of data/questions.jsonl, the consumer app (whatever is downstream of the pipeline) switches. The dashboard's download zip starts including data/split/*.jsonl. The existing data/questions.jsonl continues to be written for backward compatibility.

Phase 8 — Future possibility (not part of this plan)

- If the user later wants to parallelize the three passes across machines, the manifest-based incremental join (G.3) becomes relevant. The current Phase 7 is sufficient for the in-process shared-state design.

Summary

The current architecture is NOT safe to split at the Gemini-call level. The shared `chapter_records` in-memory state is load-bearing for 14+ invariants, including the run-18 hallucination guard, the carry-forward expiry, the section-boundary hard-reset, the integrity sweep's duplicate-stem check, and the contamination guards. Splitting the calls would silently lose all of these.

The architecture IS safe to split at the write level — after all in-pipeline reconciliations are done, write the chapter's already-reconciled records to three sidecar JSONL files (`questions.jsonl`, `answers.jsonl`, `solutions.jsonl`), keyed by a `q_id` that is a composite of deterministic anchors (printed stem, printed solution header, answer-key row, OCR, neighbor run, carry origin, section position). Each row carries a `q_id_grade` (`RESOLVED_ANCHORED` / `RESOLVED` / `PROVISIONAL` / `UNRESOLVED`) computed from the anchors. A separate `unresolved_qids.jsonl` records every `q_id` that could not be reconciled. The original `data/questions.jsonl` is untouched, so `qbank_validator.py` and the dashboard are not affected.

A consumer that joins the three split files on `q_id` gets the same logical row it gets today, plus a clear, deterministic signal of which rows are confident and which are not. The user's 8 edge cases are all handled, with the explicit guarantee that **no record is silently attached to a different question** when identity cannot be established.

The recommended implementation is strictly additive and goes in 8 phases, each gated on a manual review of the trial book.

Ready for your implementation instruction. Please confirm whether you want to proceed with **Phase 1** (passive observer of `q_no_anchors`) or with a different starting point, and whether the contract described in Section D matches what you intended.